Individual Software Project Proposal

Web Application Security Scanner for Detecting SQL Injection, XSS, and CSRF

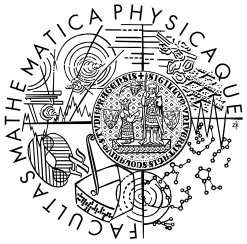
Author: Hasan Alizada

Faculty: Faculty of Mathematics and Physics

Supervisor: RNDr. Michal Kopecký, Ph.D.

Study Programme: Computer Science

Prague, 2025



FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

Contents

Chapter 1

Introduction

1.1 Overview of the Project

Web applications are widely used and therefore frequent targets for attackers. This project implements a Linux-friendly command-line scanner, written in Python, that automates detection of three common vulnerabilities: SQL Injection (SQLi), Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF).

1.2 Scope

Testing and validation will be performed only in authorized, local test environment (DVWA). The tool focuses on detection and easy extensibility to add new checks.

Chapter 2

Project Approach

2.1 Design Principles

The scanner follows a simple modular design:

- **Discovery:** crawl and enumerate pages, parameters and forms.
- **Payload engine:** context-aware payloads for GET/POST/headers.
- **HTTP client:** configurable requests with cookie support.
- **Detection:** heuristics combining content, error messages and timing.

2.2 Implementation Notes

- Language: Python
- Key libraries: `requests/httpx`, `beautifulsoup4`, `pytest`.
- CLI: single script entry point, module selection via flags.
- Configuration: simple file or command-line options.
- Safety: rate limiting and tests only in controlled environments.

Chapter 3

SQL Injection (SQLi)

3.1 Definition

SQLi occurs when untrusted input alters an application's database query logic, allowing unauthorized data access or manipulation.

3.2 Attack Patterns

- Logical bypass (e.g. `' OR 1=1--`)
- Error-based disclosure
- UNION-based data injection
- Blind (boolean/time) techniques

3.3 Detection Strategy

1. Discover inputs (query strings, form fields, headers).
2. Send short test payloads and check if the page echoes them.
3. Check responses for obvious error messages or test markers.
4. If needed, compare two requests (different payloads) to see if the output or timing changes.
5. Report only when two independent checks match to avoid false positives.

Chapter 4

Cross-Site Scripting (XSS)

4.1 Definition

XSS allows attackers to inject script into pages viewed by other users, enabling session theft, UI manipulation, and other browser-side attacks.

4.2 Types

- Reflected — payload echoed in response.
- Stored — payload persists on the server and affects others.
- DOM — vulnerability occurs in client-side script handling.

4.3 Detection Strategy

- Locate input reflection and context (HTML, attribute, script).
- Use context-appropriate payloads and markers.
- For DOM XSS, use lightweight rendering or heuristic checks to detect risky sinks.
- Prefer evidence of reflection/execution before marking high severity.

Chapter 5

Cross-Site Request Forgery (CSRF)

5.1 Definition

CSRF tricks a logged-in user's browser into performing actions unknowingly by leveraging automatic cookie sending.

5.2 Detection Strategy

- Identify state-changing endpoints (POST/PUT/DELETE; GET with side effects).
- Check forms for anti-CSRF tokens (hidden fields, headers).
- Verify presence of origin/referer checks and `SameSite` cookie attributes.
- In authorized tests, verify whether replayed requests without tokens are accepted.

Chapter 6

How to Run the Project

6.1 Setup Instructions

1. Install Python 3.10+ and clone the repository.
2. Create a virtual environment and install dependencies:

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

3. Run DVWA locally and obtain a valid session cookie.

6.2 Usage Examples

Replace `<PHPSESSID>` with your session id.

XSS

```
python3 scanner.py \
  --url "http://127.0.0.1/dvwa/vulnerabilities/xss_r/?name=test" \
  --cookies "PHPSESSID=<PHPSESSID>; security=low" \
  --modules xss -v
```

SQL Injection

```
python3 scanner.py \
  --url "http://127.0.0.1/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" \
  --cookies "PHPSESSID=<PHPSESSID>; security=low" \
  --modules sqli -v
```

CSRF

```
python3 scanner.py \
  --url "http://127.0.0.1/dvwa/vulnerabilities/csrf/" \
  --cookies "PHPSESSID=<PHPSESSID>; security=low" \
  --modules csrf -v
```

Chapter 7

Evaluation and Testing

7.1 Manual Testing

Tested against DVWA:

- SQLi: error-based and union-based detection on the `id` parameter.
- XSS: reflected payloads in the `name` parameter; form-based checks for stored types.
- CSRF: detection of missing tokens on state-changing form endpoints.

7.2 Automated Testing

Unit tests (via `pytest`) cover:

- CSRF heuristics
- Payload injection helpers
- Basic response parsing and detection functions

7.3 Example Results

The scanner successfully identified reflected XSS and SQLi in DVWA with clear payload/evidence reporting. CSRF checks flagged forms lacking anti-CSRF tokens.

Chapter 8

Resources and References

Libraries and Tools

- `requests` / `httpx` — HTTP requests
- `beautifulsoup4` — HTML parsing
- `pytest` — tests
- DVWA — Vulnerable Application

OWASP Resources

- SQL Injection
- Testing for SQL Injection — WSTG
- SQL Injection Prevention Cheat Sheet
- XSS Prevention Cheat Sheet
- Cross-Site Request Forgery (CSRF)
- Testing for CSRF — WSTG
- CSRF Prevention Cheat Sheet
- OWASP Top 10 Proactive Controls
- OWASP Cheat Sheet Series: Index of Proactive Controls

Other Security Blogs and Tools

- Cloudflare: OWASP Top 10 Overview
- Snyk: Implementing OWASP Top 10 Proactive Controls

Academic Resources

- Stanford University: Web Application Security Lecture Slides